Assignment 4 – Automated track infrastructure recognition using vibration analysis

Participants

Tor Suneson. All by myself.

Source code available at GitHub - torsuneson/Assignment4-public: D7015B Industrial AI and eM · GitHub

AI Statement

I used Microsoft Copilot as a programming assistant for debugging, code suggestions, and refactoring. All final design decisions, parameter choices, and evaluation/interpretation of results were performed by the author. VSCodium with Pyrefly was used for code completion.

Since this is not a programming course, and that it has been stated that it ´s OK to use AI for help, that's how I have approached analysing the data.

CoPilot was also used for improving some sections of the report where it was very hard to express the actions taken in an academic language. Therefore, the quality of the text might vary throughout the document.

Index

Assignment 4 – Automated track infrastructure recognition using vibration analysis 1
Participants 1
AI Statement 1
Background 4
Dataset: 4
Instructions for assignment 4
Setup of the accelerometers 5
Introduction 6
Methods 6
Mapping, Grade 3 6
Labelling and classification (Grade 4 and 5) 8
GPS refinement 8
Vibration refinement 8
Features 8
Time 10
Feature selection 11
Classical models 12
Deep learning 12
Results 13
Classical models 13
Initial configuration 13
Improved data set selection (spline) 15
Deep learning 18
Initial configuration 18
Improved data set selection (spline) 20
Keras deep learning 21
Keras POC 22
Discussion 25
Inference 26
Reference literature 26

Background

"In this study, we installed a suite of sensors to collect high-resolution data from train movements, which includes both rail vibration and Global Positioning System (GPS) measurements. Two rail vibration sensors were mounted on the train, Sensor 1 on the left side and Sensor 2 on the right, providing complementary vibration signals. The GPS unit simultaneously recorded latitude, longitude, and speed, while also logging the number of connected satellites to assess data quality. In addition to sensor data, infrastructure event information (including turnouts, joints, and bridges) was sourced from the primary control system. However, these events are not exclusively associated with the track currently used by the train; they sometimes correspond to nearby sub-tracks, necessitating an additional filtering step to isolate events that the train actually encountered.

The main goal of our project is to develop a robust algorithm capable of detecting track infrastructure events, specifically turnouts, joints, and bridges, directly from the rail vibration signals. This capability would facilitate more effective and automated monitoring of track conditions in real time." -From instructions.

Dataset:

Data 1: reference points for Bridge, RailJoint, Turnout in Longitude and latitude coordinates.

Data 2: 15 GB of raw data that includes accelerometer data from two accelerometers (Left and right channel), GPS data, number of GPS satellites connected and GPS speed.

NOTE!

I just noticed that, this being at the very end of the lab, that the folder structure for Data 2 is a mess. Sometimes the data is inside a different folder with the same name as the folder that was just entered. This means that my script for selecting runs probably did not work as intended. But being so fa into the project now it seems hard to go back and redo everything. In a real world scenario, that's what would have happened but now I will just try to slightly improve the results.

Instructions for assignment

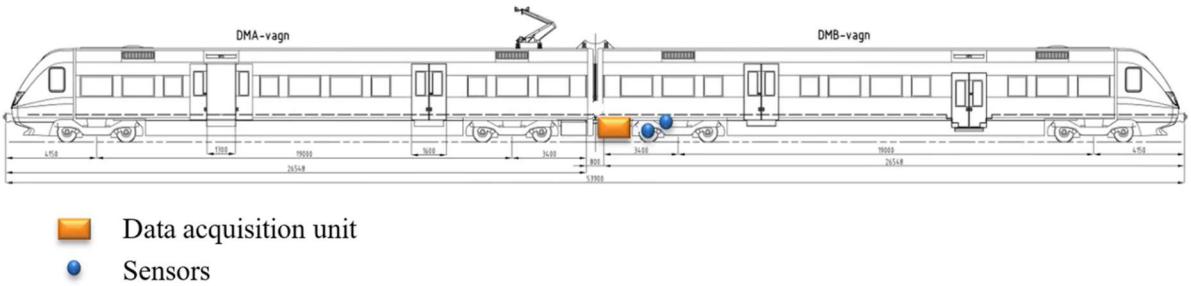
Tasks to complete:

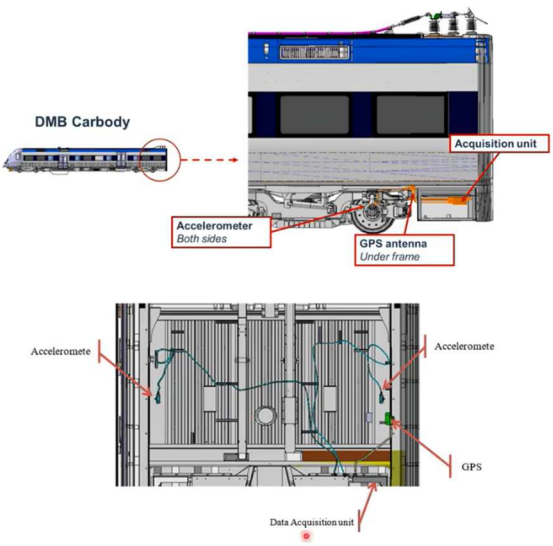
1. **Grade 3 : Mapping**
    - Complete Code 1, load the three CSV files (From Data 1 folder), draw the map, save a static image. **Explain.**
    - Using Code 2, select only the GPS/VIB files data (From Data 2 folder) that belongs to the same track in code 1, draw & save images. **Explain.**
2. **Grade 4 : Labelling**
    - Keep only the points actually travelled (Bridge, RailJoint, Turnout).
    - Label every vibration segment with the corresponding categorical label

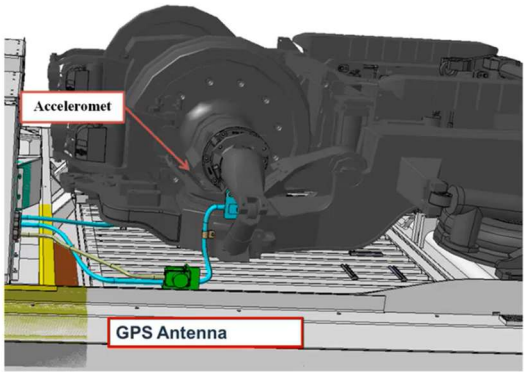
(Bridge, RailJoint, Turnout, and other).

3. **Grade 5 : Classification**
    a. Extract numerical features from every vibration segment.
    b. Train several classical ML models and at least one deep-learning model.
    c. Report and compare the metrics, keep the best model.

Setup of the accelerometers







The data was collected along real railway tracks on trains moving from Mora to Borlänge, and also out to Falun.

Introduction

Implementation evolved in four stages.

1. Mapping scripts were completed to visualize Data 1 reference points and Data 2 travelled GPS trajectories.
2. A run-selection step was introduced to filter the large Data 2 archive into a reproducible manifest of runs matching the reference track.
3. Data refinement and feature extraction were implemented to improve GPS trajectory quality and to compute segment-level vibration features.
4. A unified training pipeline was created to generate weak labels, reuse Assignment 3 feature selection, train multiple classical models, and run an MLP baseline with consistent cross-validation.

Methods

The work follows a structured pipeline: map and filter the correct travel track (Grade 3), create weak labels for vibration segments using Data 1 reference locations (Grade 4), and train and evaluate classifiers on segment-level features using a leakage-aware split strategy (Grade 5).

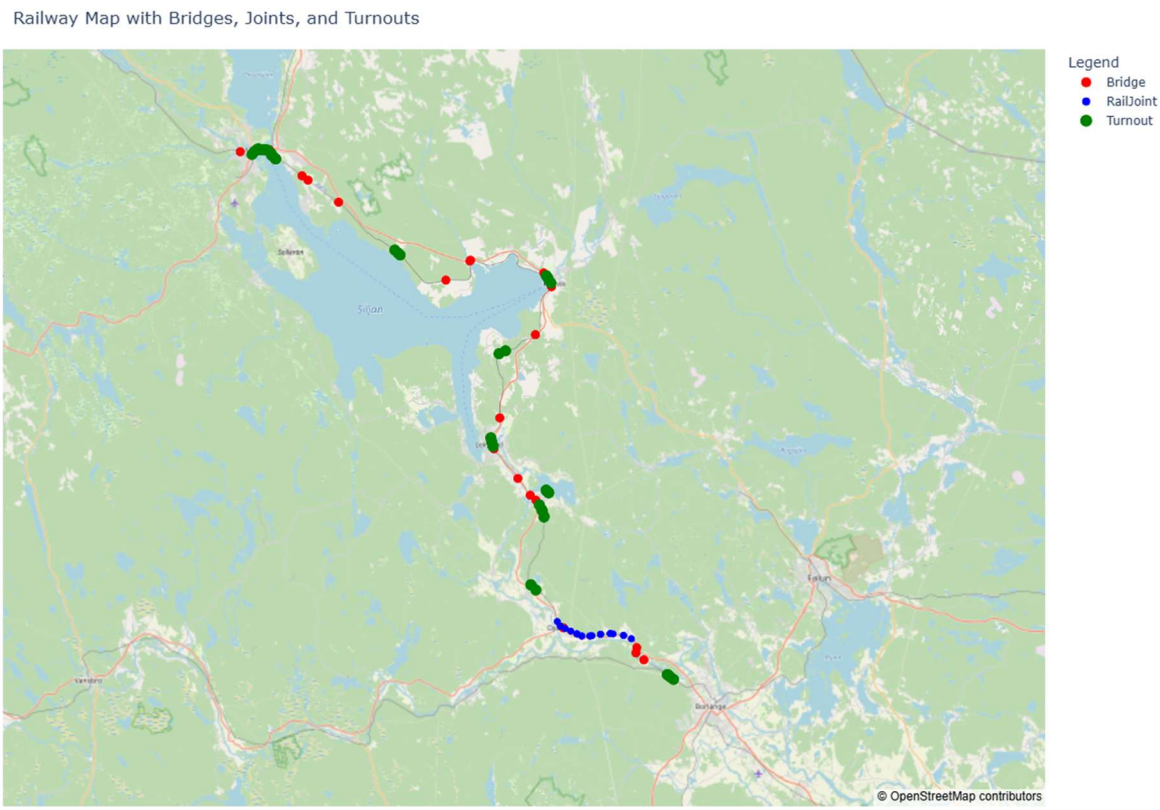
Each stage reduces uncertainty: track filtering reduces irrelevant reference events, weak labeling enables supervised learning on Data 2, and GroupKFold-by-run prevents overly optimistic results caused by dependence between neighbouring segments.

Mapping, Grade 3

I completed the code in `Code 1-1.ipynb` so it reads the data from “Data 1” folder and saves a map as PNG. For easy usage I also export a HTML-file that can be loaded into a browser for zoom and movement.

I had to change from `scatter_mapbox` to `scatter_map` to make export work. Because when `scatter_mapbox` was used the map ended up Africa all the time, lat/long and center/zoom function did not accept given values. `Scatter_map` changed the way centre could be calculated and the map ended up in correct place and correct zoom level.

Result is saved as `railway_map.html` and `railway_map.png`.



For the second part of grade 3 I did not get `Code 2-1.ipynb` working without modification, so I dumped the python code in a file named `Code2.py`. There were also parts of the instruction that is unclear how to proceed with.

Using Code 2, select only the GPS/VIB files data (From Data 2 folder) **>that belongs to the same track in code 1<**, *draw & save images.* ***Explain.***

I also had a hard time finding out which data out of the 15GB that travelled along the track between Mora and Borlänge. And rendering them all so the maps could be inspected I felt would take too long time.

Because Data 2 contains many runs across different routes, it is necessary to first isolate runs that correspond to the same geographic corridor as the Data 1 reference track. The manifest-based approach ("`selected_runs.json`") provides a reproducible subset of runs, making later labeling and training consistent and computationally feasible. This also reduces label noise from reference points belonging to neighbouring lines that the train did not traverse.

For the purposes of selecting the correct dataset I made "`select_runs.py`" that when activated will traverse "Data 2" .csv files and of they score how similar GPS coordinates they have with "Data 1" and selected into a manifest for later usage. The manifest is called `selected_runs.json`

At first no data containing RailJoints where selected. For some reason the score was very low on these files, probably because there were so few RailJoints compared to the bridges and turnouts. I placed debug code to “not find” RailJoints in data set in `Code2 debug.ipynb`

With that in mind a special ENSURE_RAILJOINT_RUN was introduced into the code to get at least some RailJoints into the selected dataset.

With manifest selected the file `Code2.py,` containing most of the original functionality with some modifications, will export the map to a folder called “Code2_exports”.

To run the file a block in `code2.ipynb` need so be activated.

Labelling and classification (Grade 4 and 5)

For labelling and classification, the data needed to be refined, and features needed to be added for better classification. A helper file called `refine_data.py` was added into `select_runs.py` to see if a refined data could find more datapoints in Data 2.

GPS refinement

Filtering out invalid point was actually done for the first `select_run.py` because invalid data points stopped moving forward in dataset. Points with coordinates containing 0,0 or ouf-of-range are invalidated and removed.

For the even better data quality a jump-filter + interpolation with soothing (Exponential Moving Average, EMA) was introduced to get datapoint in gaps and also adjust datapoints that where “off” in the series.

Vibration refinement

Since I worked with telecom for 15 years and filtering out signals from noise, I use some old tricks on the vibration data that was used in telecom.

A high pass filter to subtract moving average from the signal (trend removal), mildly low-pass filtered, normalized and segmented into fixed-length windows.

Features

Data 2 lacks manual segment-level labels, so supervision is approximated by weak labeling: each segment is assigned the class of the nearest Data 1 reference point within a radius, otherwise ‘Other’. This produces a usable training target but introduces label noise, especially for short events (e.g., rail joints) and when GPS-vibration alignment is imperfect.

To be able to classify the data better at a later point features were added for each segment. Time-domain features include RMS, peak, kurtosis, crest factor, impulse factor, zero-crossing rate, spike counts.

A simple frequency-domain feature FFT band power was computed.

Weak labelling improvement.

Because segment labels in Data 2 are obtained via proximity to Data 1 reference coordinates, the labelling quality depends strongly on GPS reliability and the segment-to-GPS time mapping. To reduce label noise, GPS samples with low quality were filtered using satellite-count gating and a speed-based jump guard. When a segment mapped to an invalid GPS sample, the nearest valid sample within a small index window was used instead. This reduces systematic mislabelling caused by GPS glitches and improves the consistency of weak labels.

Short-duration event features.

RailJoint events are short, impulse-like phenomena that can easily be smoothed out when long, fixed-length segments are described using only averaged statistics such as RMS. To better capture these brief transients, several additional features were introduced, including peak-to-RMS (peak_over_rms), peak-to-peak-to-RMS (p2p_over_rms), spike density (spikes per second), and a frequency-domain band power ratio (bandpower_ratio_1) when multiple bands were available. These features were added as an exploratory measure to improve class separability for minority classes, with a particular focus on enhancing “RailJoint” detection.

“Quick” data selection

After running into persistent issues with weak labelling, and getting the sense that the selected runs were not really representative, I decided to revisit the data selection step. I implemented a function that fits a spline along the track using the Data 1 reference coordinates and then selects Data 2 runs whose GPS trajectories align with this spline. Unfortunately, this approach made the selection even stricter, and the number of usable runs dropped further. After some threshold tuning, I ended up with 14 selected runs, which already felt like too much fine-tuning for comfort.

Instead of trying to *über-match* the GPS trajectories to the track geometry, I switched to a simpler “quick selection” approach. This method uses a geographic bounding box combined with a running-distance constraint (50–120 km) to filter the dataset. With this strategy, trips to Falun and short station runs were likely excluded. Using this approach, the selection was reduced to seven runs. Both “quick” and “spline” selection will later in the report be tested against the initial configuration.

The “Quick” selection resulted in a more balanced distribution of events overall, although the proportion of the “Other” class is still noticeably larger, with a factor of almost 20 times compared to “RailJoint”

"Quick" selection:

Label distribution:	**35998**
Other	29862
Turnout	3131
Bridge	1510
RailJoint	1495

The larger selection algorithm, with tuning, selects 14 runs, and is still heavy on "Other" events.

By "Spline" selection:

Label distribution:	**100791**
Other	54772
Turnout	34472
Bridge	9774
RailJoint	1773

Time

Because Data 2 lacks explicit ground-truth labels per segment, I used Data 1 reference coordinates to weakly label segments within a distance radius and applied a direction-dependent time shift to reduce systematic misalignment between GPS and vibration.

Visualization of the data shows if the data collection was done northbound or southbound.

I tried many different methods to get the GPS data to better align with Data 1 points, but I did not get them to work properly. So, I left it as it was and focused on labelling the data instead.

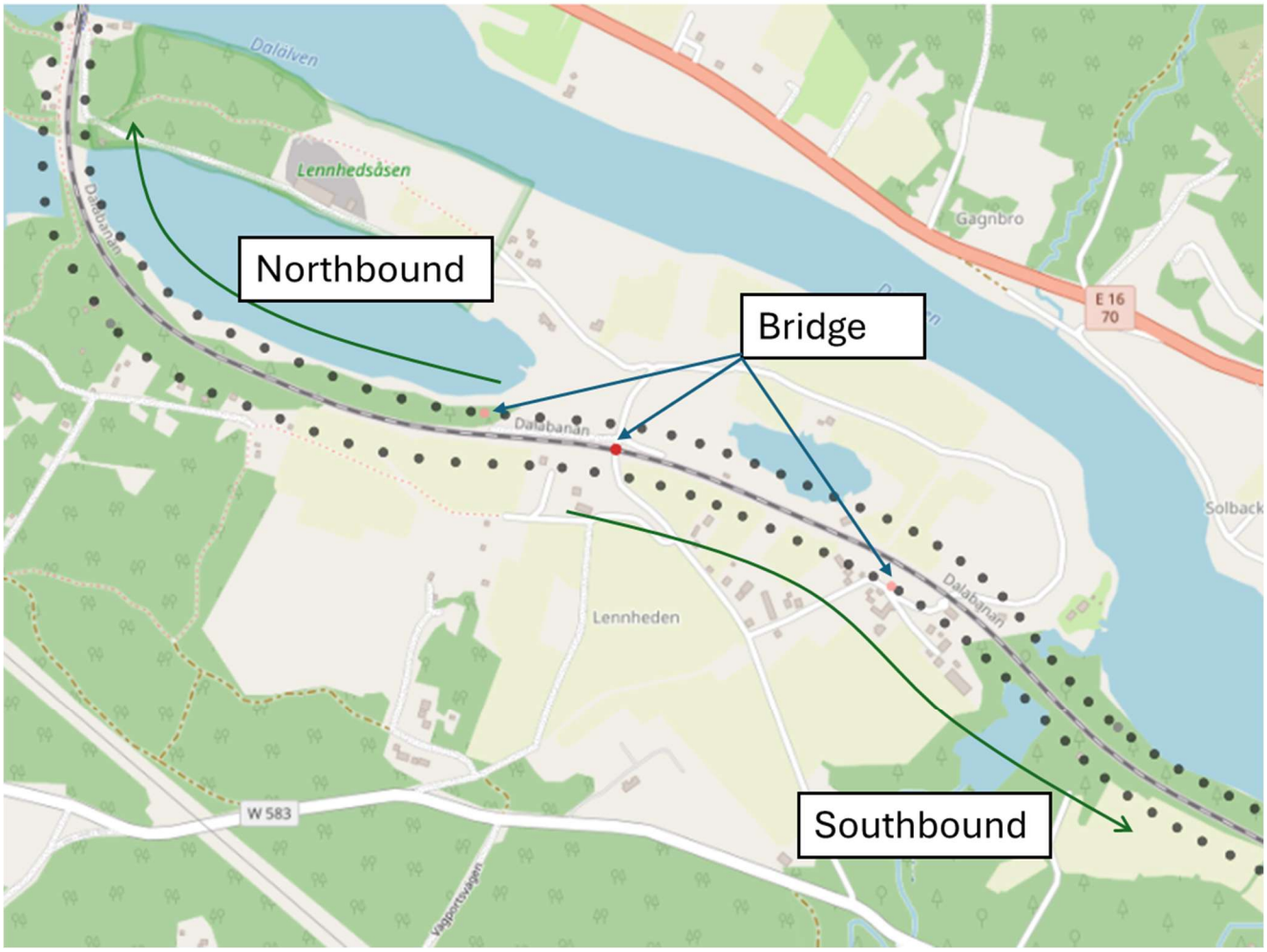


Figure 1: GPS data collection is directional and therefor labelling of segment will not be aligned with map. Directions in example above are guessed based on that events are label after real world event due to time shift.

Because GPS and vibration are not perfectly time-aligned in Data 2, a direction-dependent time shift was used to compensate for systematic offsets. This improves the spatial alignment between detected events and Data 1 reference locations, but it was hard to get a correct alignment. A test of using overpass railway data was made to "tie a event" to a location and by that calibrate the segments. But due to complexity in the API and that I had "fixed time" segments the results did not get any better.

Feature selection

The feature set contains correlated time-domain and frequency-domain statistics (e.g., RMS/energy/peak), making feature selection valuable for stability and interpretability. For continuity with Assignment 3, filter (InfoGain), embedded (L1), and wrapper (SFS) approaches were reused to reduce redundancy and highlight features that consistently contribute under run-based cross-validation.

I selected one method from each group in assignment 3: Filters, embedded methods and wrappers.

- Filter method is mutual gain (InfoGain) since it performed well on time-domain features.
- L1 logistic regression selection as feature selection from the embedded methods. As seen in Assignment 3 Random Forrest had a better performance for the final

classification, but for feature selection is considered more stable and reliable. It is also clean and reproducible.
- Sequential Feature Selector (SFS) from the wrappers.

In the beginning It seemed like there was leakage between the segments so GroupKFold by run was introduced to minimize the data leakage from the same run.

Segments within the same run are temporally and spatially correlated, so random splitting can leak run-specific characteristics into both train and test sets. GroupKFold-by-run ensures evaluation on unseen trips, yielding more realistic generalization estimates.

These methods are used before training to select a smaller more informative feature set.

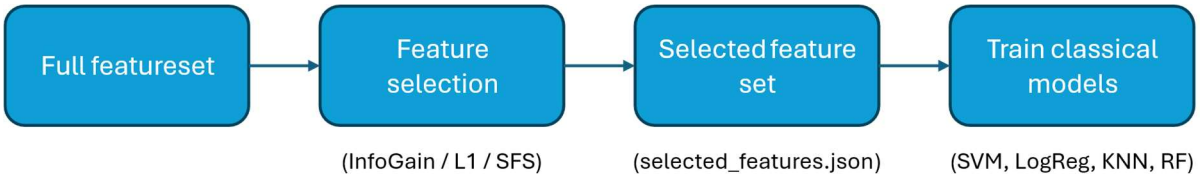


Figure 2: Process of feature selection

Classical models

Feature selection was performed using the three methods developed in Assignment 3: InfoGain (filter), L1-regularized logistic regression (embedded), and Sequential Feature Selection (wrapper). These methods reduce the dimensionality of the feature space and help remove irrelevant or redundant features.

After feature selection, several classical ML classifiers were trained (SVM-RBF, Logistic Regression, KNN (kNearest Neighbours), Random Forest). These are the actual predictive models. The feature selectors are not classifiers themselves—they prepare the feature space, but they do not perform classification.

The comparison in 'Model comparison (macro-F1)' therefore reflects the performance of the classical models after feature selection, not the feature selection algorithms themselves.

Deep learning

Since I did not have any deep learning library, I had to google geek4geek again and see what they recommended. *Cuantum Technologies* (no author listed) book [10] also gave some hints to how to proceed.

MLP is one of the simplest Deep learning neural networks pioneered by Frank Rosenblatt in the late 1950s. The book also provided a suitable starting point for using an MLP, including Python-based examples.

The MLP achieved 0.635 accuracy but macro-F1 of 0.233, indicating that performance is dominated by the majority class (Turnout). The model failed to detect "RailJoint" and "Other" (F1=0), suggesting either that the model rarely predicts these classes or that predictions are

systematically incorrect. This is consistent with strong class imbalance and potential label noise due to GPS-vibration misalignment, particularly affecting short-duration “RailJoint” events.

Results

Classical models

Initial configuration

The classical machine-learning models—SVM with RBF kernel, Logistic Regression, k-Nearest Neighbours, and Random Forest—were all trained on the same engineered feature set and evaluated using GroupKFold-by-run in order to avoid leakage between segments originating from the same physical journey. The comparison based on macro-F1 shows that all models face substantial difficulty in separating the four event classes, with overall scores ranging from approximately 0.200 to 0.277.

Macro-F1 is emphasized because the class distribution is highly imbalanced; accuracy would be dominated by the majority class and hide poor minority-class performance.

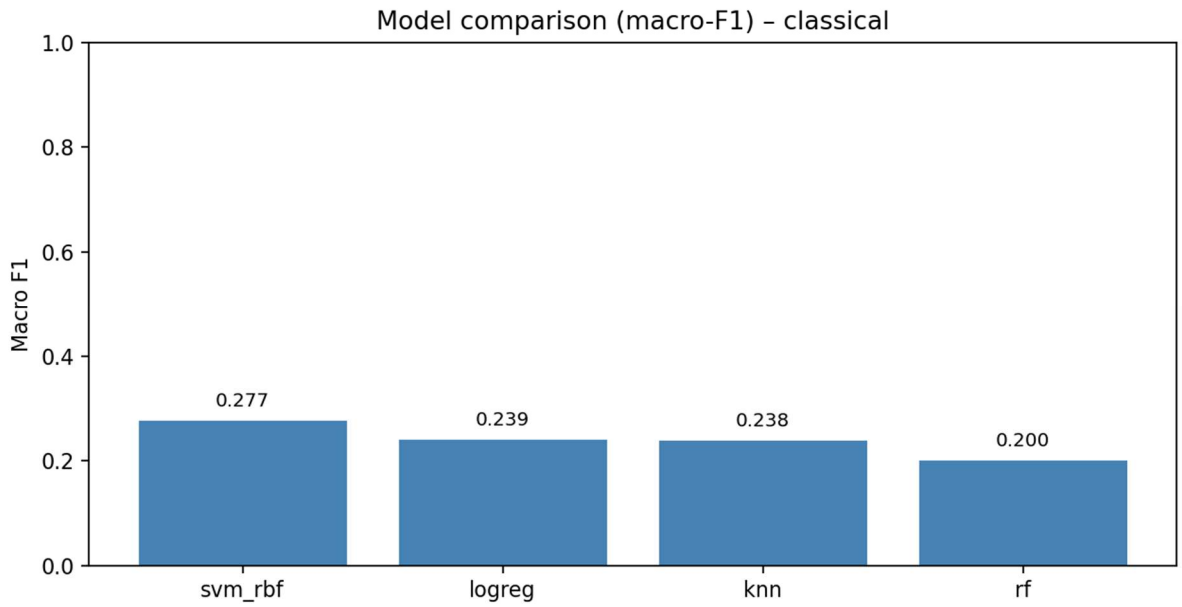


Figure 3: Macro F1 score for models shows difficulties sperating the four event classes

Among them, the “svm_rbf” achieved the highest macro-F1 score, though still modest (≈0.277). When examining the svm_rnf confusion matrix, it becomes clear why macro-F1 remains low: the model performs well on the majority class “Turnout”, correctly identifying roughly 91% of these segments, but it struggles significantly on the minority classes.

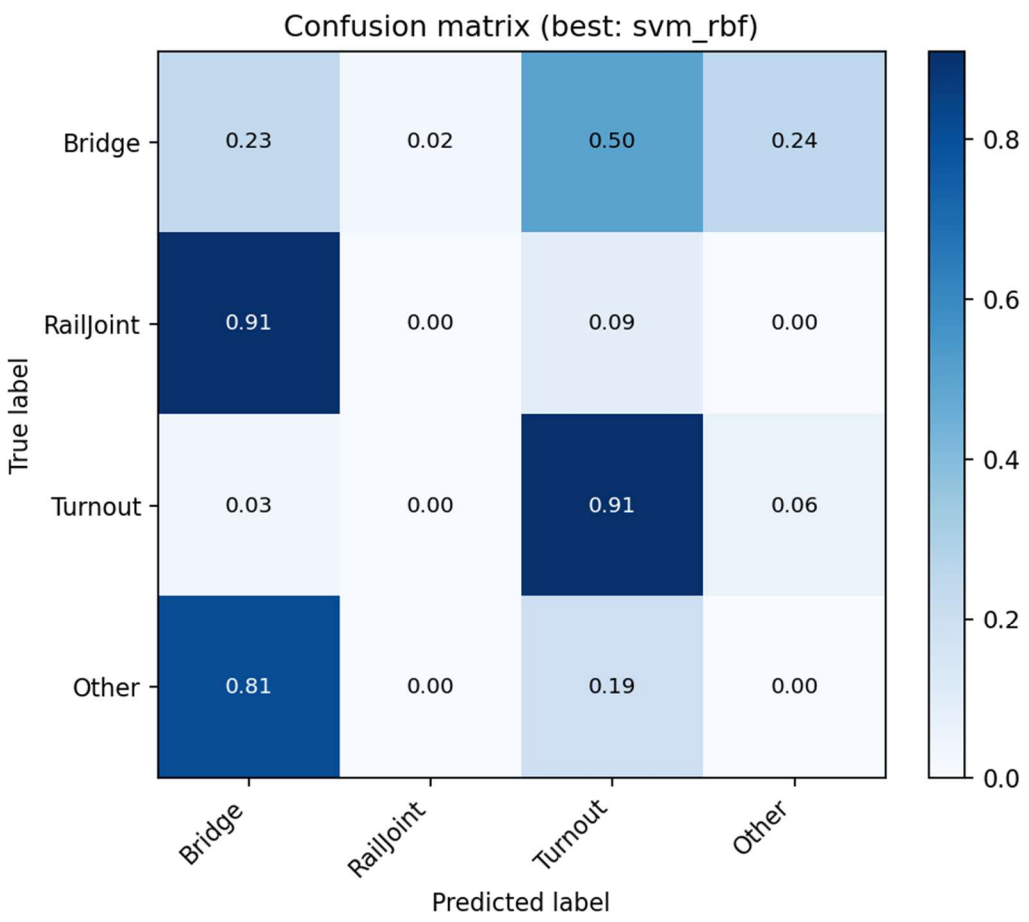


Figure 4: Confusion matrix for svm_rbf (best) model of True label and Predicted label

"RailJoint" is almost never predicted, being instead misclassified mainly as Bridge. "Other" exhibits a similar collapse. "Bridge", although more represented, is often confused with "Turnout". This pattern indicates that the SVM learns a reasonably strong decision region for "Turnout", likely because it dominates the feature space, but lacks sufficient discriminative information to reliably separate the more subtle variation between "Bridge", "RailJoint", and "Other". Logistic Regression and kNN follow similar trends with even lower performance, and Random Forest performs worst, which suggests that the features, even after filtering, L1 selection and sequential forward selection do not cluster into clean separable groups. Overall, the classical models reveal a consistent picture: the problem is not that the algorithms are underperforming but that the dataset and the weak-label structure produce inherently overlapping feature distributions for three of the four classes, limiting achievable macro-F1 regardless of classifier choice.

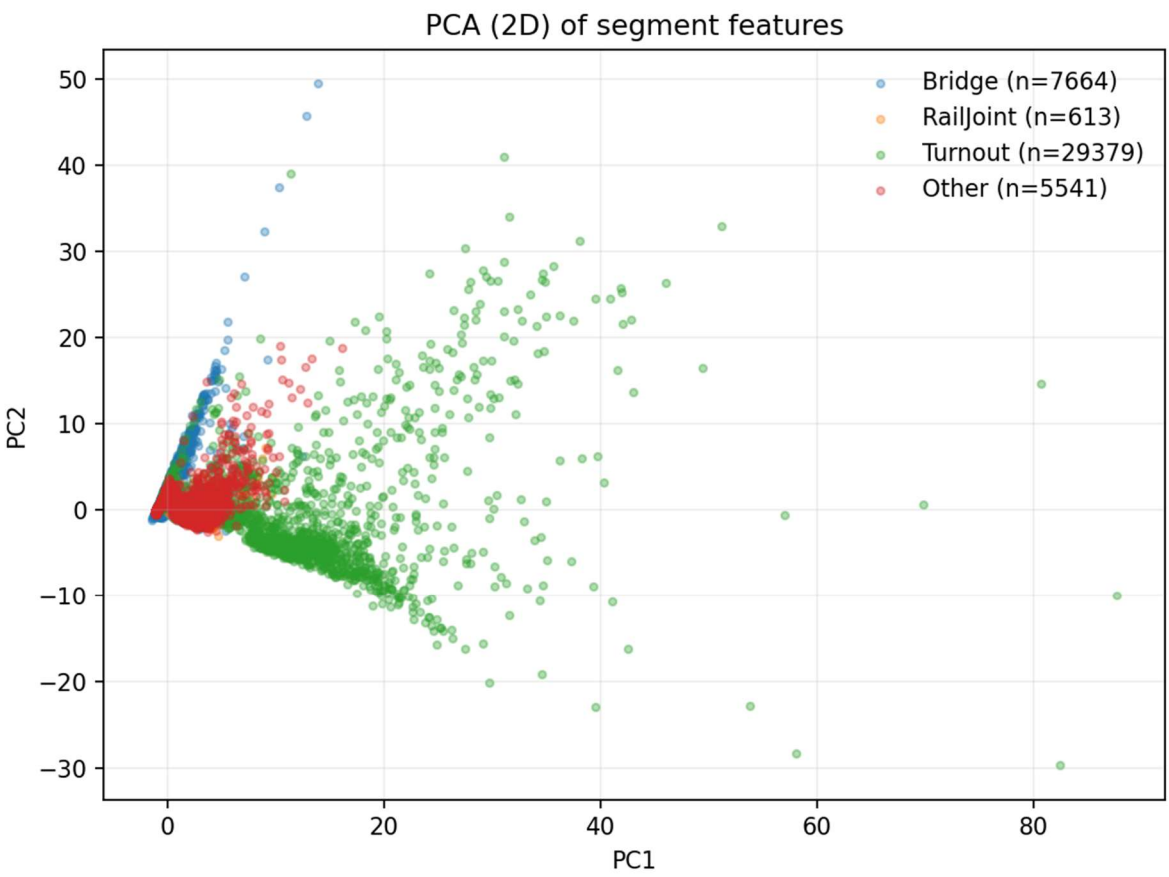


Figure 5: PCA plot of classes

The PCA visualization provides qualitative support: Turnout occupies a more separable region, while Bridge/RailJoint/Other overlap substantially, which aligns with the observed misclassifications in the confusion matrices.

Improved data set selection (spline)

It took 485 min and 26 sek to train with 100791 rows and with new set of short-duration features (peak-to-RMS (peak_over_rms), peak-to-peak-to-RMS (p2p_over_rms), spike density (spikes per second), and a frequency-domain bandpower ratio (bandpower_ratio_1) added. Still having a large bias for “Other” events, it was interesting to see what would happen.

Label distribution:	**100791**
Other	54772
Turnout	34472
Bridge	9774
RailJoint	1773

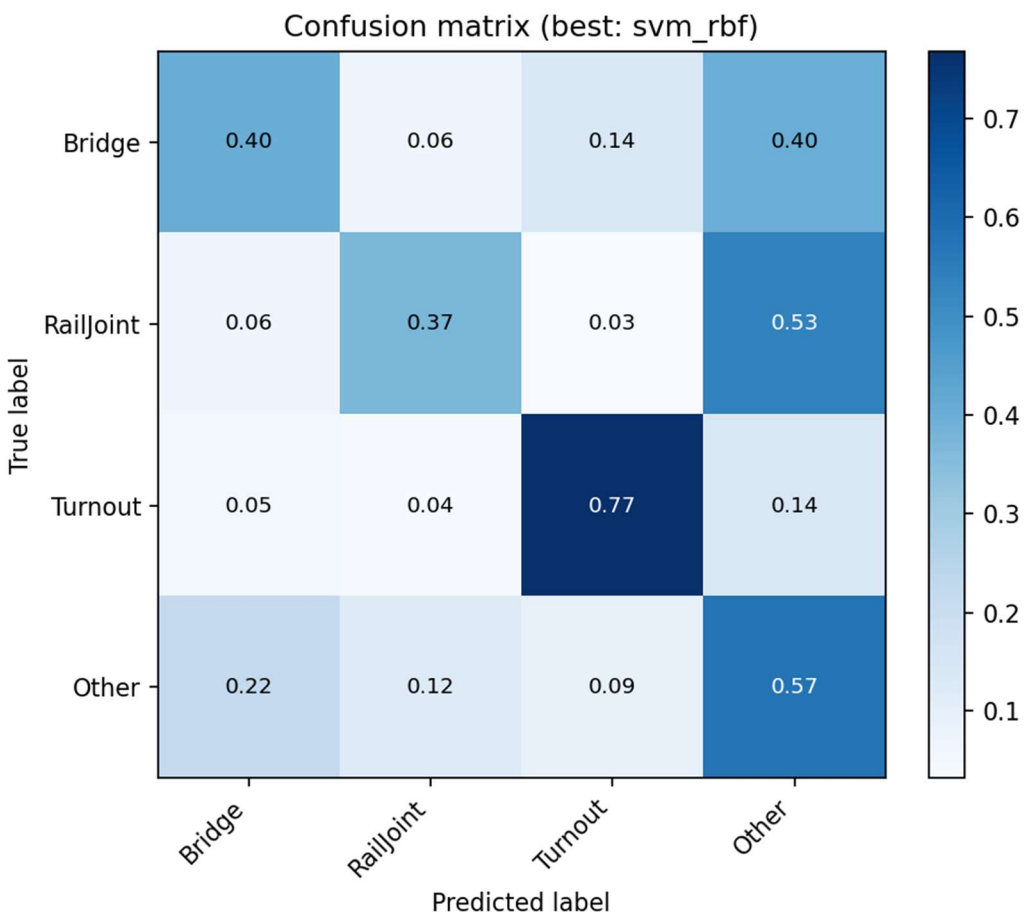


Figure 6: Confusion matrix after improved data selection algorithm

Confusion matrix now shows more of what we want to see for Support Vector Machine, Radial Bias Function kernel (svm_rbf). RailJoint still being a problem getting predicted as “other” but all other classes (Bridge, Turnout and Other) get highest score results at correct classification. So, for classic models the results have improved. Both SVM RBF and LogReg uses class_weigh “balanced”, and this is why the predictions mostly are correct.

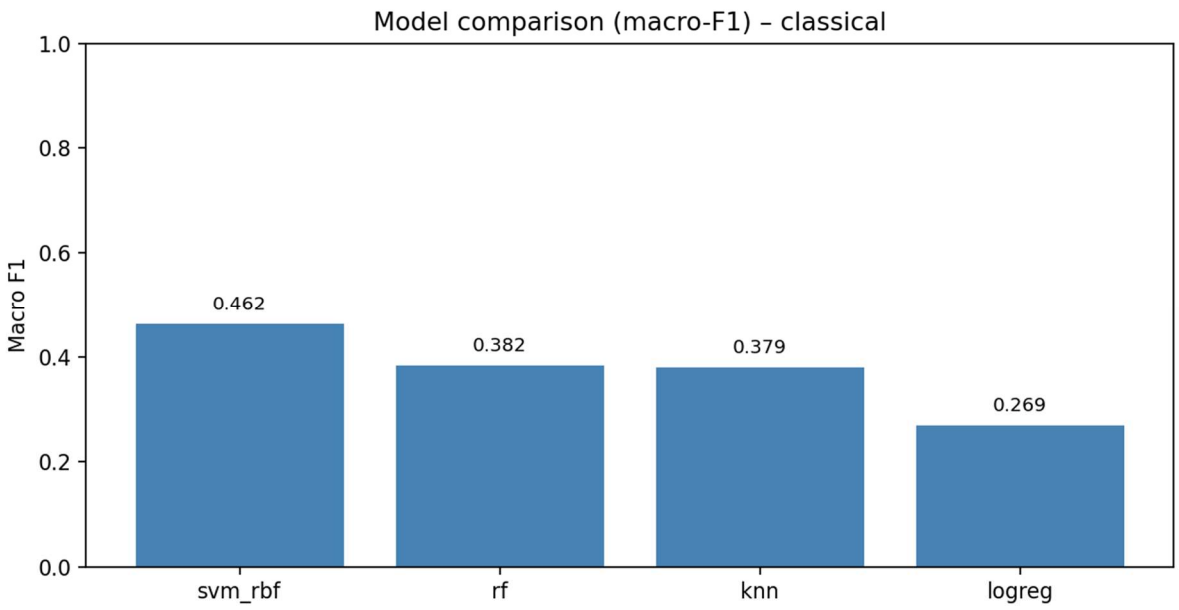


Figure 7: Macro F1 scores after improved selection algorithm.

It is also clear that the Macro-F1 score have improved for all models.

I regret changing two things at the same time, because now I don't know which change made difference. Introducing both short-duration features and changed the selection of data sets at the same time was not scientifically wise.

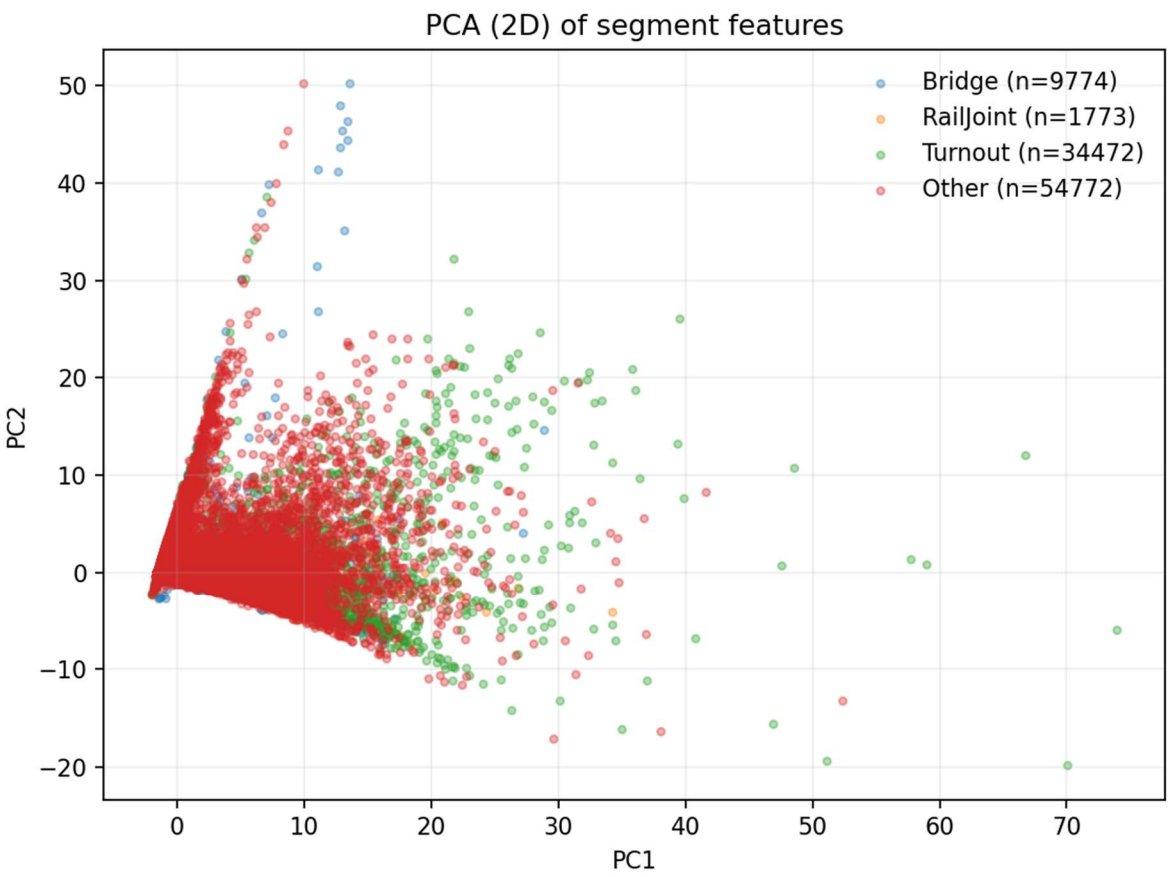


Figure 8: PCA of spline selected runs

PCA still show dominance of “Other” events and not much separation to “Bridge”, “Turnout” and “RailJoint”.

Deep learning

Initial configuration

The Multi-Layer Perceptron (MLP) was trained using the same selected features as the classical models. Although deep learning can in theory capture more complex decision surfaces, the MLP achieved performance similar to the classical methods, with a macro-F1 of approximately 0.233.

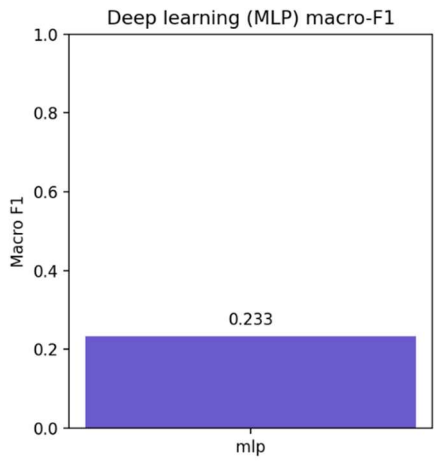
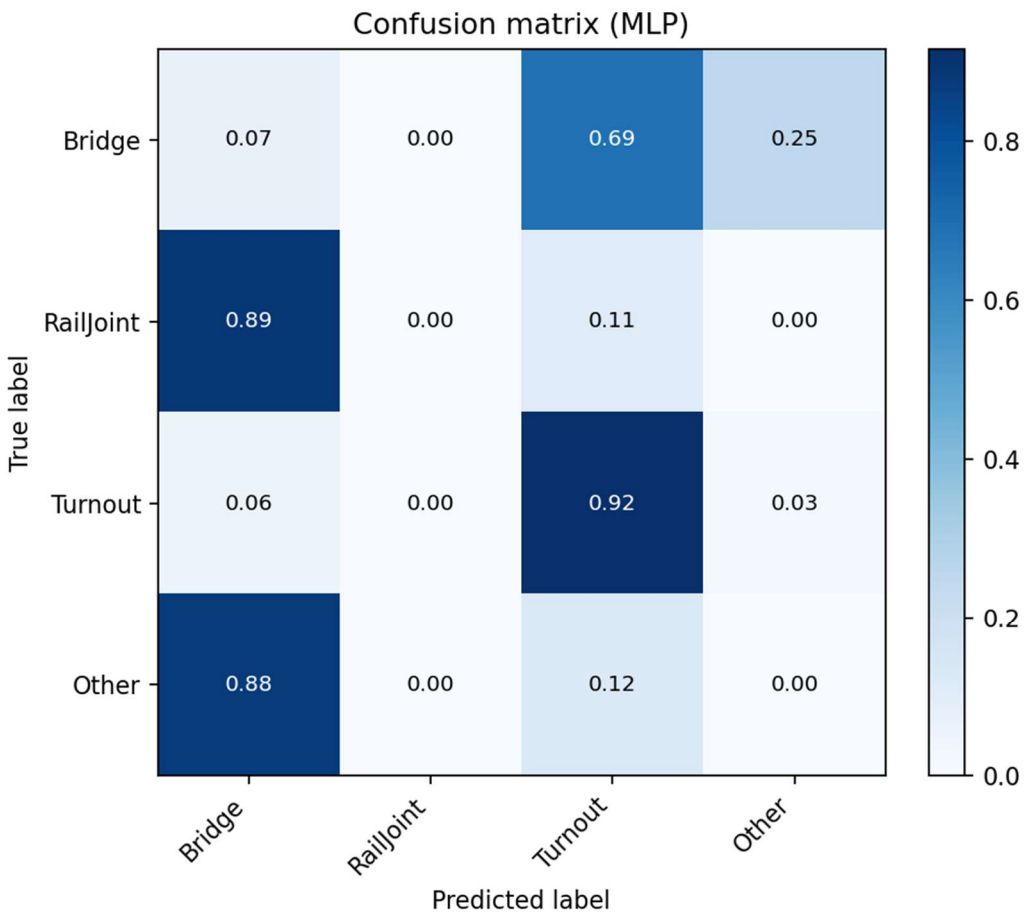


Figure 9: Macro F1 for MLP model

Its confusion matrix shows a pattern closely matching the SVM-RBF: strong identification of **Turnout**, near-complete failure on **RailJoint** and **Other**, and substantial mixing between **Bridge**, Turnout, and Other.



This indicates that the MLP, despite being a more flexible model class, is still constrained by the structure and quality of the input features and labels. Because the MLP receives only aggregated statistics from each segment—not raw vibration sequences—and because the labels come from GPS-based weak supervision, the network learns a representation that emphasises broad, dominant patterns rather than short, subtle events like RailJoints. In this sense, the MLP does

not overfit; on the contrary, it generalises almost identically to the classical models, suggesting that the underlying problem is the lack of distinct information available in the features rather than model capacity. The similarities between the SVM and MLP confusion matrices reinforce that the deep model is not uncovering new discriminative structure; instead, it converges to the same Turnout-centric solution because Turnout is the only class with clear, separable patterns given the noisy and imbalanced feature space. Thus, the MLP results should be interpreted not as a failure of neural networks but as confirmation that the current representation and label quality limit classification performance across model families.

Improved data set selection (spline)

On the other hand, the MLP seems to collapse totally not being able to predict anything other than “Other”, and very confident at that too.

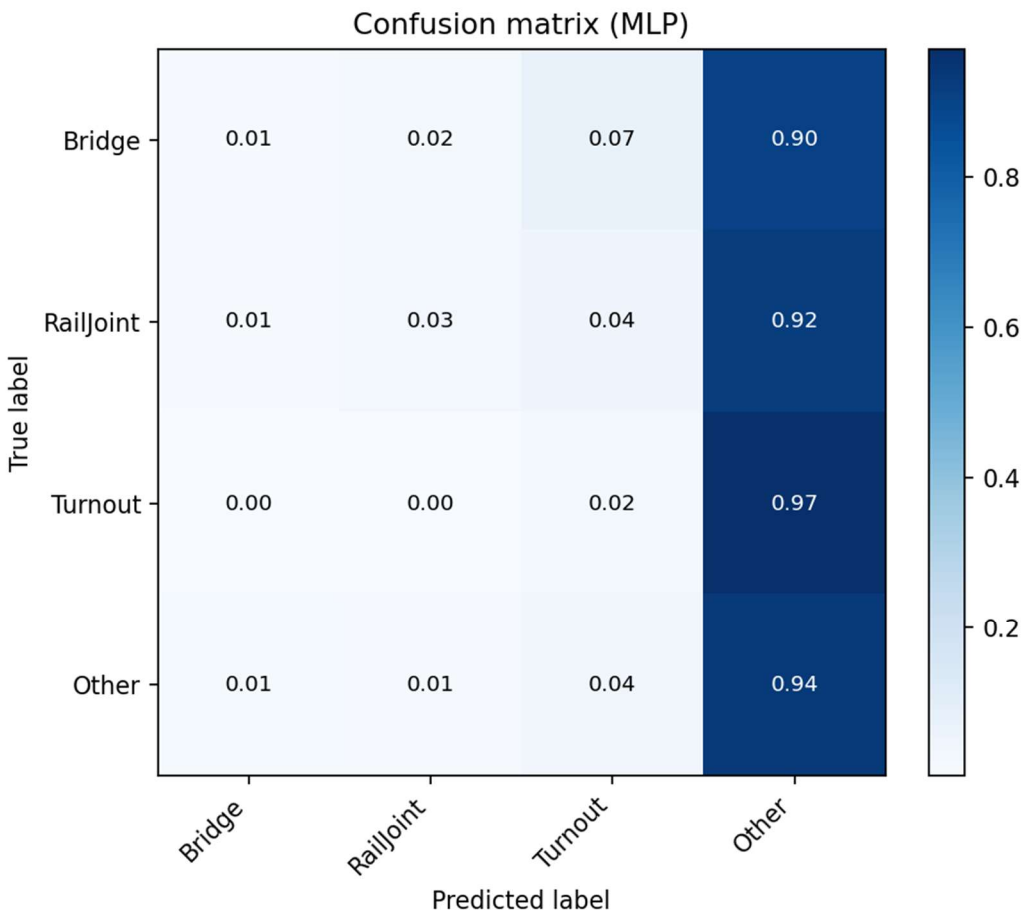


Figure 10: MLP confusion matrix after Spline selection of runs

The MLP collapsed to predicting the majority class “Other” for nearly all segments (≈90–97% across true classes in the normalized confusion matrix). This behaviour is consistent with severe class imbalance and weak-label noise. “Other” is both the largest and most heterogeneous class, and MLPClassifier does not directly support class weighting. As a result, optimizing log-loss favours a majority-class solution that yields moderate accuracy but near-zero recall for minority classes.

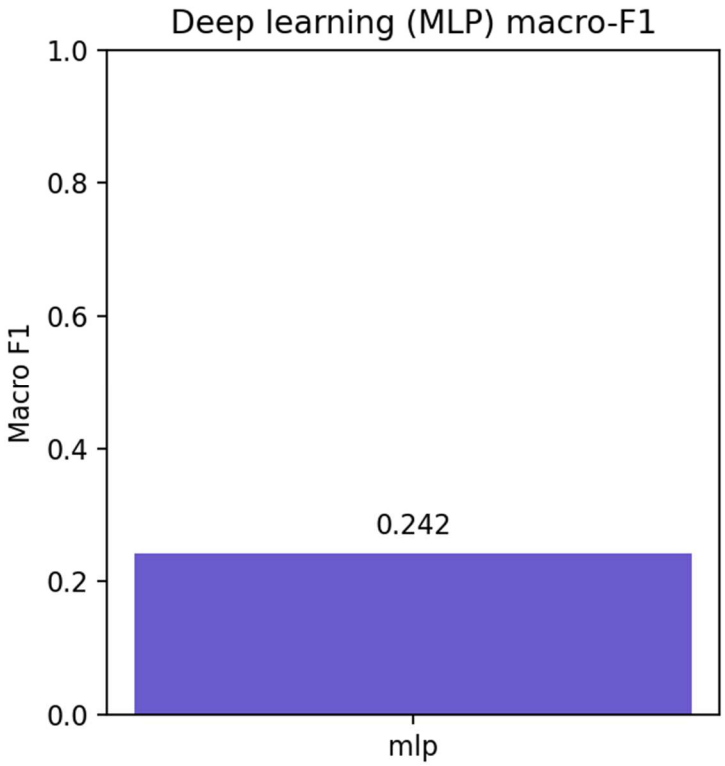


Figure 11: Macro F1 for MLP using spline selected runs

The Macro-F1 is slightly higher than the initial configuration (0.233) but still very low, and with "Other" being such a dominant class, about ~54% of all labels. This gives the model ~0.54 a "free" prediction without any training. The model then collapses due to the bias of the "Other" class and only predicts "Other" for all classes.

Keras deep learning

MLP does not consider the that the domination of "other" compared to the other Classes. LogReg and SVM RBF uses Class weight to even out dominance of one class. Bu using "balanced" input to the model it will automatically scale the classes making the difference between them more equal.

TensorFlow library offers a deep learning called Keras and it is described as a "A Corner stone in deep learning community" [10, chaper 3].

Since the results were so bad with the MLP model I really wanted to try out a deep learning that would work. Keras has the ability to "balance" classes and thereby automatically scale biased classes so the balance becomes more equal. This is the deep learning model I should have tried from the beginning. By using the examples in [10] **Cuantum Technologies**, *AI Mastery Series: Book 2: Deep Learning and AI Superhero: Mastering TensorFlow, Keras, and PyTorch,* First edition, ISBN: 979-8-89587-359-5 I put together another file called `keras_mlp_weighted.py` that implement a drop in supplement for the MLPClassifier.

Traing on the same `selected_segments.csv` and the changes dramatically

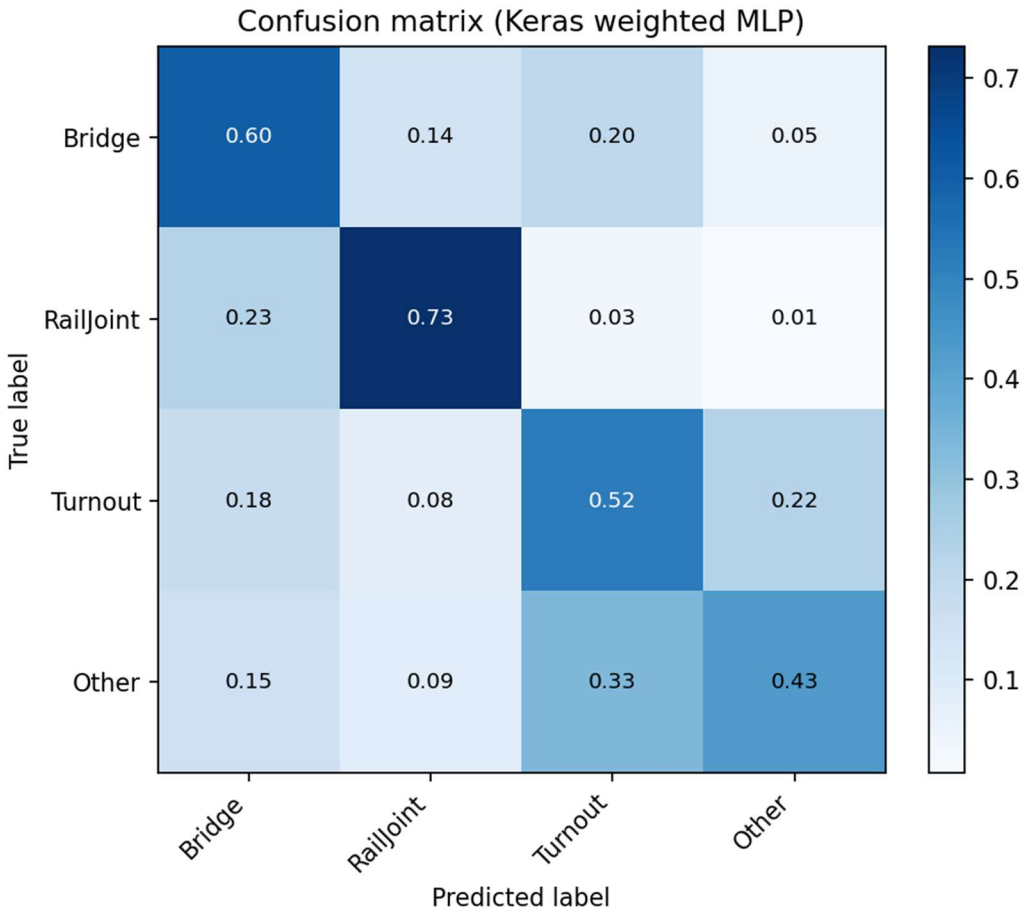


Figure 12: Keras model after using the same selected_segmets.csv ad as for MLPClassifier

Now "Bridge", "RailJoint", "Trunout" and "Other" all have the highest score when predicted. This is the best result of all we have seen.

Keras POC

Now I wanted to see if the prediction works! I manually (I could have done this from the beginning saving med to have to write `select_runs.py`). Then manually creating a quick_maifest.json and going ahead to segment and label the runs.

I found that there where several runs passing though Örebro, going down to Mjölby that could work for the POC. So I saved the model into `keras_mlp_weigheted.keras` and `keras_scaler.npz` after the model had been trained. According to (my now favourite book) **Cuantum Technologies**, *AI Mastery Series: Book 2: Deep Learning and AI Superhero: Mastering TensorFlow, Keras, and PyTorch,* First edition, ISBN: 979-8-89587-359-5 [10] I could not use the same approach when I had a saved model. I had to create a "inference" segmented data set with features detected in the vibration signals. In the helper lib "label.py" a function named `build_inference_dataset` does just that, build the inference dataset.

" AI inference is the execution phase. It uses the trained and fine-tuned model to make fast predictions on new, "unseen" data. Each individual prediction is far less computationally

demanding than training but delivering millions of predictions in real-time requires a highly optimized and scalable infrastructure." Source Google

Not having any clue what I'm doing, just heading out in space the function was more or less from "copied" the book and combined with the some of the features that we previous had.

Run	Segment	Latitud	Longitud	Features
run	seg	Latitued	Longitud	peak_over_rms, p2p_over_rms, spike_density, bandpower_ratio_1

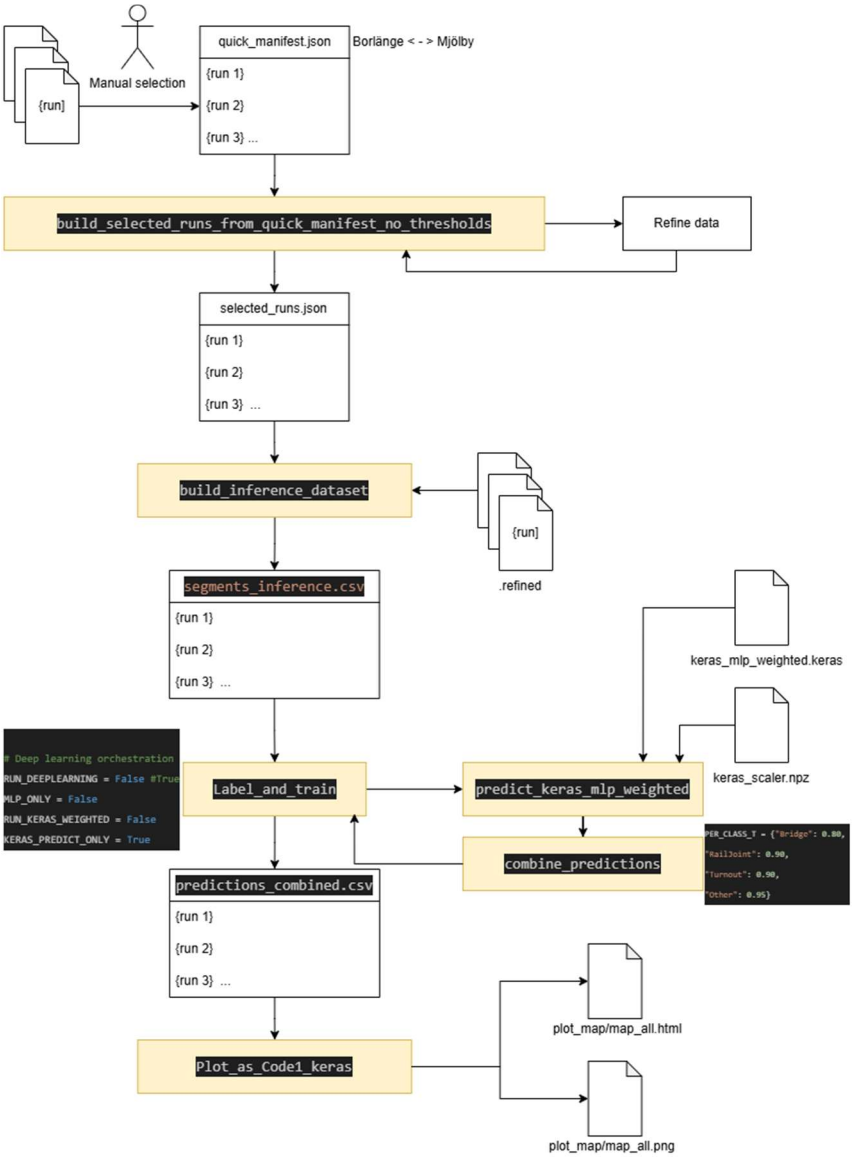


Figure 13: Inference of the keras MLP model on railway between Borlänge and Mjölby

This was not a straightforward process and I'm not sure it actually predicts anything useful. At first the all the GPS points got a prediction, and I tried to combine the dataset to get better results, but I think my late-night work only combined the different runs in a file, not combine the data for better results... Well. Plotting the results also showed long runs of the same prediction,

mostly “others” but also “Turnout” seem to dominate.

Figure 14: Map of predicted classes.

In an effort to reduce the number of predicted classes I introduced a stage in `ensemble_combine.py` that thresholds the predictions, and if it’s below the threshold it is set to “Neurtral”.

Predicted class	Threshold
Bridge	0.80
RailJoint	0.90
Turnout	0.90
Other	0.95

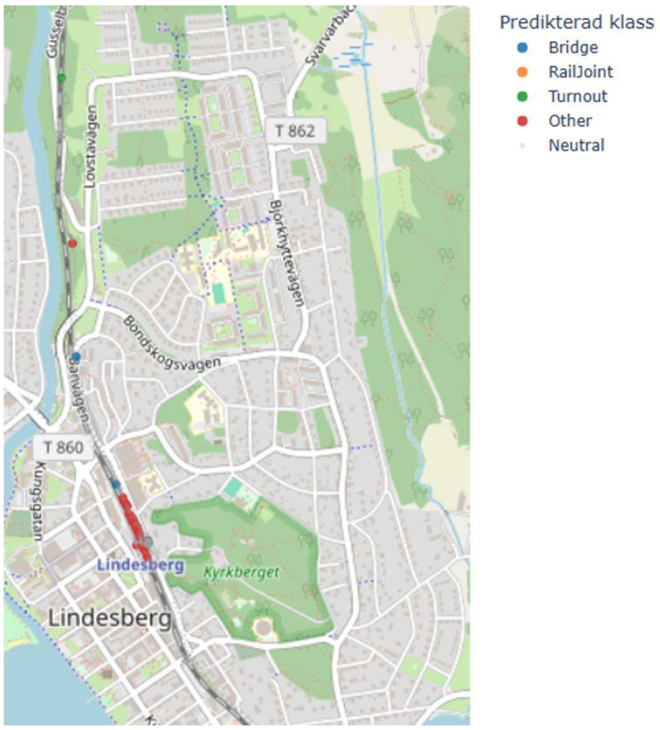


Figure 15: Predictions after thresholds

Hard to say if the predictions work or not. This is clear, at railway stations the predictions become more frequent. This indicates that the model finds known vibration-patterns at these stations. And Bridges seems also to predict quite well, even though there off in GPS drift. Turnouts and RailJoints is not possible to verify without more data.

Discussion

Several realistic improvements could raise performance if the project were extended. The most impactful would be addressing the quality of the weak labels. The current labelling depends on aligning GPS points with vibration segments using fixed direction, dependent time shifts and proximity radius. Short-duration events like RailJoints are extremely sensitive to even small misalignments, so improved alignment, using, for example, dynamic time-shift estimation, cross-correlation with known landmarks such as bridges, or probabilistic distance-based labelling, could significantly reduce noise in minority classes.

A second concern is class imbalance, where oversampling RailJoint and Other, or applying cost-sensitive learning, would help shift the model away from its current Turnout-dominant predictions. Improvements to the feature extraction pipeline would also matter: using shorter segmentation windows, multi-resolution windows, or adding high-frequency impulse-sensitive features (such as wavelet coefficients or peak-envelope metrics) could make RailJoints more distinguishable. Using a weighted class when train models (SVM RBF ang LogReg) improved results dramatically. The MLP classification did not have support for weighted class so to train a deep learning model Keras MLP in TensorFlow library was used. With weighted class applied to Keras MLP Deep learning results became quite balanced and inspired inference of the trained model on unknown data.

Finally, a more advanced modelling strategy, such as training a hierarchical classifier (first Turnout vs non-Turnout, then finer classification), or treating RailJoint detection as a binary

anomaly-type task, might better reflect the structure of the problem. While these enhancements not perused right now, due to not enough time, they represent meaningful directions that could substantially improve Macro-F1, particularly for the minority classes that currently dominate the error profile.

Inference

I ran the trained Keras model in inference mode on an unseen railway segment to gauge real-world behaviour. The pipeline, feature extraction, scaling, prediction, ensemble on GPS-gridded points, and map visualization, shows that the model can highlight plausible events while leaving low-confidence locations as Neutral after applying a confidence threshold. Results are strongly driven by data quality and GPS refinement; with cleaner inputs, broader and more representative training data, and some additional tuning, the model's precision and spatial consistency are likely to improve.

Reference literature

1. **Paul Barry** - Headfirst Python. 3rd ed. *A Learner's Guide to the Fundamentals of Python Programming*-O'Reilly (2023), ISBN: 978-1-492-05129-9
2. **Ben Auffarth** - *Artificial Intelligence with Python Cookbook, Proven recipes for applying AI algorithms and deep learning techniques using TensorFlow 2.x and PyTorch 1.6-Packt Publishing* (2020), ISBN 978-1-78913-396-7
3. **Denis Rothman** - *Transformers for Natural Language Processing 2nd ed. Build, train, and fine-tune deep neural network architectures for NLP with Python, PyTorch, TensorFlow, BERT, and GPT-3*, ISBN 978-1-80324-733-5
4. **Aman Kedia, Mayank Rasu** - *Hands-On Python Natural Language Processing, Explore tools and techniques to analyze and process text with a view to building real-world NLP applications*, Packt Publishing Ltd (2020), ISBN 978-1-83898-959-0
5. **Sayan Mukhopadhyay** - *Advanced Data Analytics Using Python with Machine Learning, Deep Learning and NLP Examples*, Apress (2018) ISBN-13 (electronic): 978-1-4842-3450-1
6. **Saurabh Chandrakar, Dr. Nilesh Bhaskarrao Bahadure** - *Python for Everyone, Learn and polish your coding skills in Python*, BPB Publications (2023), ISBN 978-93-5551-815-6
7. **Deepali R Vora, Gresha S Bhatia** - *Python Machine Learning Projects, Learn how to build Machine Learning projects from scratch*, BPB Publications (2023), ISBN 978-93-89898-27-9
8. **Trevor Hastie, Robert Tibshirani, Jerome Friedman,** *The Elements of*
9. Statistical Learning - Data Mining, Inference, and Prediction 2nd ed., Springer, ISBN 9780387848570 https://hastie.su.domains/ElemStatLearn/ (free download)
10. **Cuantum Technologies***, AI Mastery Series: Book 2: Deep Learning and AI Superhero: Mastering TensorFlow, Keras, and PyTorch,* First edition, ISBN: 979-8-89587-359-5